

★★★ 반드시 내 것으로 ★★★

#MUSTHAVE

실무 중심 이론, 실무 중심 프로젝트로 단단하게 시작하자

성낙현의 JSP 자바 웹 프로그래밍



우리는
가치가 성장하는
시간을
만듭니다.



[1단계] 빠르게 익히는 JSP 기초

00 개발 환경 구축

- 01 JSP 기본
- 02 내장 객체(Implicit Object)
- 03 내장 객체의 영역(Scope)
- 04 쿠키(Cookie)
- 05 데이터베이스
- 06 세션(Session)
- 07 액션 태그(Action Tag)
- [Project] 08 모델1 방식의 회원제 게시판 만들기
- [Project] 09 게시판에 페이징 기능 넣기

[2단계] 고급 기능으로 스킬 레벨업

- 10 표현 언어(EL : Expression Language)
- 11 JSP 표준 태그 라이브러리(JSTL)
- 12 파일 업로드 및 다운로드
- 13 서블릿(Servlet)
- [Project] 14 모델2 방식(MVC 패턴)의 자료실형 게시판 만들기

[3단계] 프로젝트로 익히는 현업 스킬

- [Project] 15 웹소켓으로 채팅 프로그램 만들기
- [Project] 16 SMTP를 활용한 이메일 전송하기
- [Project] 17 네이버 검색 API를 활용한 검색 결과 출력하기
- [Project] 18 배포하기

Chapter

03

내장 객체의 영역(Scope)

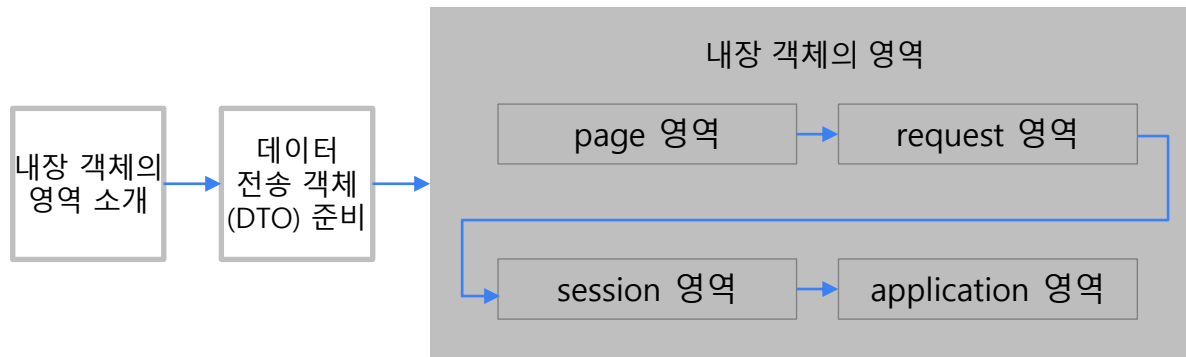


□ 학습 목표

□ 학습 순서

□ 활용 사례

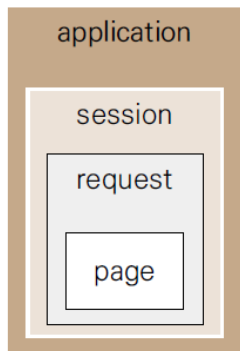
내장 객체의 유효기간이라 할 수 있는 영역과 데이터의 저장, 공유 학습



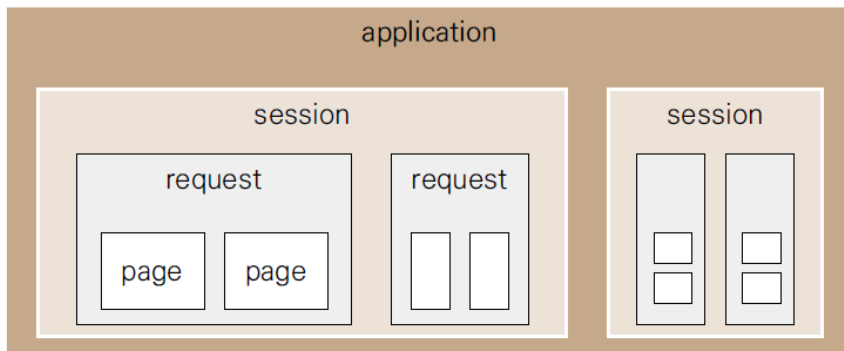
- 4가지 내장 객체의 영역 개념
웹에서는 페이지(page)들이 모여 하나의 요청(request)을 처리하며, 요청들이 모여 하나의 세션(session)을, 다시 세션들이 모여 하나의 웹 애플리케이션(application)을 구현
- 구체적인 구현 방식은 JSP와 다르더라도, 이 개념들은 다른 웹 프로그래밍 기술들에도 그대로 적용

3.1 내장 객체의 영역이란?(1)

- 내장 객체의 영역은 각 객체가 저장되는 메모리의 유효기간
 - **page 영역** : 동일한 페이지에서만 공유됩니다. 페이지를 벗어나면 소멸
 - **request 영역** : 하나의 요청에 의해 호출된 페이지와 포워드(요청 전달)된 페이지까지 공유
 - **session 영역** : 클라이언트가 처음 접속한 후 웹 브라우저를 닫을 때까지 공유
 - **application 영역** : 한 번 저장되면 웹 애플리케이션이 종료될 때까지 유지



〈범위〉



〈더 넓은 영역은 작은 영역을 둘 이상 포함 가능〉

3.1 내장 객체의 영역이란?(2)

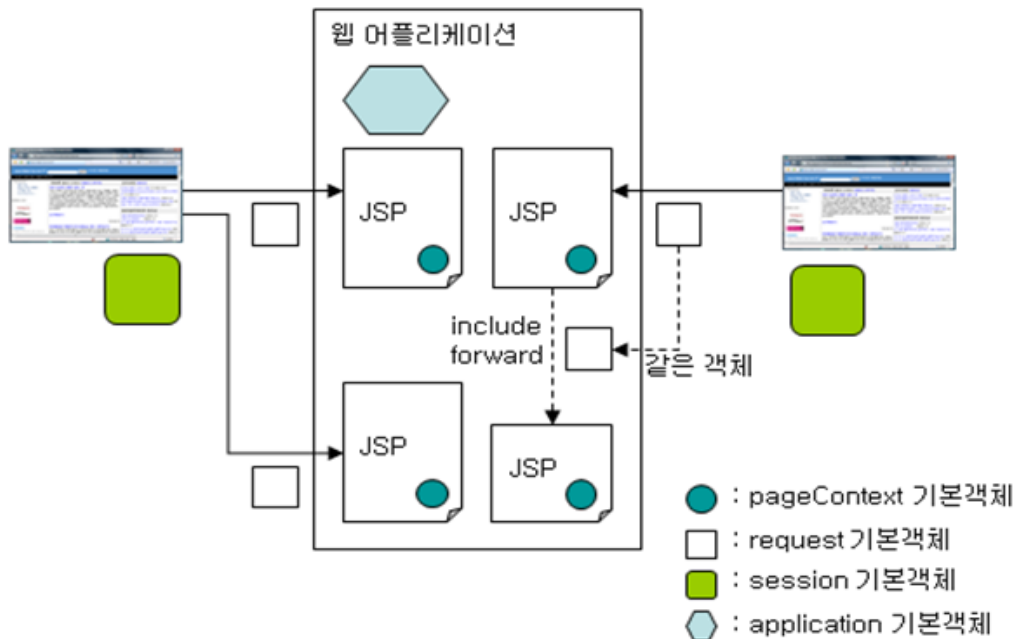
- 영역이 제공하는 주요 메서드
 - **void setAttribute(String name, Object value)**
 - 각 영역에 속성을 저장
 - 첫 번째 인수는 속성명, 두 번째 인수는 저장할 값
 - 값의 타입은 Object이므로 모든 타입의 객체를 저장
 - **Object getAttribute(String name)**
 - 영역에 저장된 속성값을 얻어옴
 - Object로 자동 형변환되어 저장되므로 원래 타입으로 형변환 후 사용
 - **void removeAttribute(String name)**
 - 영역에 저장된 속성을 삭제
 - 삭제할 속성명이 존재하지 않더라도 에러는 발생하지 않음

JSP의 4개 영역 (scope)

4가지 영역

- **Page** 영역: 하나의 JSP 페이지를 처리할 때 사용 되는 영역
- **Request** 영역: 하나의 HTTP 요청을 처리할 때 사용 되는 영역
- **Session** 영역: 하나의 웹브라우저와 관련된 영역
- **Application** 영역: 하나의 웹어플리케이션과 관련된 영역

그림)



JSP 기본 객체의 속성

- 기본 객체 pageContext, request, session, application은 속성을 갖는다.
- 속성은 각각의 기본 객체가 존재하는 동안에 사용 가능
- JSP 페이지 사이에서 정보를 주고 받거나 공유하기 위한 목적으로 사용
- 속성은 <속성이름, 값>의 형태를 가짐
- 서로 다른 이름을 가진 속성을 여러 개 포함할 수 있다.

[네 기본 객체의 속성 관련 메소드]

메소드	리턴타입	설명
setAttribute(String name, Object value)	void	이름이 name인 속성의 값을 value로 지정한다.
getAttribute(String name)	Object	이름이 name인 속성의 값을 구한다. 지정한 이름의 속성이 존재하지 않을 경우 null을 리턴한다.
removeAttribute(String name)	void	이름이 name인 속성을 삭제한다.
getAttributeNames()	java.util.Enumeration	속성의 이름 목록을 구한다. (pageContext 기본 객체는 이 메소드를 제공하지 않는다.)

☞ 속성의 값 타입은 Object이므로 값을 사용할 때에는 알맞은 형변환이 필요

속성의 활용

기본 객체	영역	쓰임새
pageContext	PAGE	(한번의 요청을 처리하는) 하나의 JSP 페이지 내에서 공유될 값을 저장한다. 주로 커스텀 태그에서 새로운 변수를 추가할 때 사용된다.
request	REQUEST	한번의 요청을 처리하는 데 사용되는 모든 JSP 페이지에서 공유될 값을 저장한다. 주로 하나의 요청을 처리하는 데 사용되는 JSP 페이지 사이에서 정보를 전달하기 위해 사용된다.
session	SESSION	한 사용자와 관련된 정보를 JSP 들이 공유하기 위해서 사용된다. 사용자의 로그인 정보와 같은 것들을 저장한다.
application	APPLICATION	모든 사용자와 관련해서 공유할 정보를 저장한다. 임시 디렉토리 경로와 같은 웹 어플리케이션의 설정 정보를 주로 저장한다.

3.2 데이터 전송 객체(DTO) 준비(1)

- 데이터 전송 객체(Data Transfer Object, DTO)
 - 데이터를 저장하거나 전송하는 데 쓰이는 객체로, 다른 로직 없이 순수하게 데이터만을 담고 있으며, '값 객체(Value Object)라고도 함
 - 자바빈즈(Java Beans) 규약에 따라 작성
 1. 자바빈즈는 기본(default) 패키지 이외의 패키지에 속해야 함
 2. 멤버 변수(속성)의 접근 지정자는 private으로 선언
 3. 기본 생성자가 있어야 함
 4. 멤버 변수에 접근할 수 있는 게터(getter)/세터(setter) 메서드가 필요
 5. 게터/세터 메서드의 접근 지정자는 public으로 선언

3.2 데이터 전송 객체(DTO) 준비(2)

- 'Person' 이름의 DTO를 작성
 - 01 프로젝트 탐색기 ① Java Resources → ② src에서 마우스 우클릭 → ③ [New] → ④ [Class]를 클릭
 - 02 ① 패키지명을 "common", ② 클래스명을 "Person"으로 입력한 후 ③ [Finish] 버튼을 클릭
 - Person 클래스가 생성
 - 03 String 타입의 이름(name)과 int 타입의 나이(age) 멤버 변수를 private으로 선언
 - 04 기본 생성자를 정의
 - [예제 3-1] Person 클래스(ver 0.1)

3.2 데이터 전송 객체(DTO) 준비(2)

- 'Person' 이름의 DTO를 작성
 - 이클립스의 자동 생성 기능을 이용
- 05 ① 편집창에서 마우스 우클릭 → ② [Source] → ③ [Generate Getters and Setters...] 메뉴 선택
- 06 [Select All] → [Generate]를 클릭
- 자바빈즈 규약을 모두 만족하는 Person 클래스가 완성
- 인수들을 받아 모든 속성을 한 번에 초기화해주는 생성자 추가
- 07 편집창에서 마우스 우클릭 → [Source] → [Generate Constructor using Fields...] 메뉴 선택
- 08 ① [Select All] → ② [Generate]를 클릭합니다
- [예제 3-2] Person 클래스(완성 버전)

3.3 page 영역(1)

- page 영역은 클라이언트의 요청을 처리하는 데 관여하는 JSP 페이지마다 하나씩 생성
 - 이때 각 JSP 페이지는 page 영역을 사용하기 위한 pageContext 객체를 할당
 - [예제 3-3] page 영역에 값을 저장하고 불러오기
 - ① 먼저 코드에서 외부 클래스인 common.Person을 사용하기 위해 임포트
 - ② pageContext 객체를 통해 page 영역에 속성값을 저장. 객체가 아닌 기본 타입 값들은 해당 래퍼 클래스로 오토박싱된 후 저장
 - ③ 저장한 속성들을 다시 읽어오기. 단, 모든 속성이 Object 타입으로 저장되어 있으므로 다시 원래의 타입으로 형변환
 - ④ String 타입인 경우 toString() 메서드를 통해 문자열로 변환하여 출력할 수도 있음
 - ⑤ Person 객체에서 멤버 변수의 값을 읽어오는 예. Person은 DTO라서 멤버 변수가 private으로 선언되었으므로 게터 메서드를 이용
 - ⑥ include 지시어로 다른 JSP 파일을 포함. '포함' 관계이므로 '같은 페이지'
 - ⑦ 에서 <a> 태그로 감싼 링크를 클릭하면 '다른 페이지'로 '이동'. 따라서 이전 페이지에서 만든 page 영역은 소멸

3.3 page 영역(2)

- [예제 3-4] PageContextMain.jsp에 포함시킬 JSP 문서
 - ① 앞의 [예제 3-3]에서처럼 page 영역에 저장된 속성을 읽어와서 형변환
 - ② 직접 출력
- [예제 3-5] <a> 태그 링크로 이동할 별도 페이지
 - 다른 페이지로 이동한 후 page 영역이 공유되는지 확인
 - ① page 영역에서 속성값을 가져오기. 이번에는 형변환을 하지 않음
가져오려는 속성이 존재하지 않는다면 `getAttribute( )` 메서드가 `null`을 반환하고, `null`을 `int` 타입 변수에 담으려 시도하면 `NullPointerException`이 발생하기 때문
 - ② 대신 값을 화면에 출력할 때 `null`이 아닌지 확인

3.4 request 영역(1)

- request 영역은 하나의 요청에 대한 응답이 완료될 때 소멸하게 되므로 page 영역보다는 접근 범위가 조금 더 넓음
 - [예제 3-6] 최초 페이지(request 영역 동작 확인용)
 - ① request 영역에 String 객체와 Person 객체를 저장
 - ②와 ③ request 영역에 저장된 속성을 삭제
 - ② "requestString"은 당연히 정상적으로 삭제되는데,
 - ③ 두 번째 줄에서는 존재하지 않는 "requestInteger"를 삭제하려 시도
 - ④ 속성값을 읽어와서 출력
 - ⑤ RequestForward.jsp로 포워드하는 코드
 - ⑥ request 내장 객체를 통해 실제로 포워드를 수행하는 코드

```
request.getRequestDispatcher("포워드할 파일 경로").forward(request, response)
```


3.4 request 영역(2)

- 포워드된 페이지에서도 영역이 공유되는지 확인
 - [예제 3-6]에서 포워드하는 부분의 주석을 해제
 - getRequestDispatcher()의 반환 타입이 RequestDispatcher이며, 이 객체가 요청을 다른 페이지로 넘겨주는 기능을 수행
- [예제 3-7] 포워드되는 JSP 페이지
 - ①과 ② request 영역에 저장된 속성들을 읽어와서 출력
 - ③ 인코딩 방식을 UTF-8로 변경
 - ④ 포워드하면서 쿼리스트링으로 전달한 매개변수의 값을 출력
 - [예제 3-6]의 RequestMain.jsp를 다시 실행하여, 포워드된 후에도 request 영역이 여전히 유효한지 확인
- request 영역에 저장된 속성값은 현재 페이지와 포워드된 페이지까지 공유

3.5 session 영역(1)

- 세션(session): 클라이언트가 서버에 접속해 있는 상태 혹은 단위, 주로 회원인증 후 로그인 상태를 유지하는 처리에 사용
 - [예제 3-8] 최초 페이지(session 영역 동작 확인용)
 - ① ArrayList 컬렉션을 생성한 후 2개의 String 객체를 저장
 - ② 이 컬렉션을 통째로 session 영역에 저장
 - ③ session 영역이 이동된 페이지에서도 공유되는지 확인하기 위한 링크
 - [예제 3-9] 링크를 클릭해 이동된 페이지
 - ① session 영역에서 속성을 읽어온 후 형변환. 컬렉션의 타입은 ArrayList<String>
 - ② for문을 이용해 컬렉션에서 객체들을 꺼내 출력

3.5 session 영역(2)

- [예제 3-8]의 SessionMain.jsp를 실행
- [SessionLocation.jsp 바로가기] 링크를 클릭
 - 페이지가 이동되었지만 session 영역에 저장된 속성값은 정상적으로 출력
 - session 영역의 속성값을 삭제하고 싶다면 웹 브라우저를 완전히 닫았다가 다시 열면 됨
 - 탭만 닫아서는 session이 삭제되지 않고, 반드시 웹 브라우저 전체를 닫아야 함
- 웹 브라우저 전체를 닫은 후 [예제 3-9]를 다시 실행
 - 500 에러가 발생
 - 웹 브라우저를 닫으면 session 객체가 삭제되고, 웹 브라우저를 다시 실행하면 그때 새로운 session 객체가 생성
 - `getAttribute("lists")`로 속성값을 읽어오려 하면 null을 반환하여 `NullPointerException`이 발생

3.6 application 영역(1)

- 웹 애플리케이션은 단 하나의 application 객체만 생성하고, 클라이언트가 요청하는 모든 페이지가 application 객체를 공유
 - application 객체는 웹 서버를 시작할 때 만들어지며, 웹 서버를 내릴 때 삭제
 - [예제 3-10] 최초 페이지(application 영역 동작 확인용)
 - ① HashMap 컬렉션을 생성한 후 두 개의 Person 객체를 저장
 - ② 컬렉션 채로 application 영역에 저장
 - [예제 3-11] 결과 페이지
 - ① [예제 3-10]에서 application 영역에 저장한 "maps" 속성값을 읽어서 원래 형태인 Map<String, Person> 타입 변수에 저장
 - Map 컬렉션에 담긴 데이터를 확인하려면 먼저 키(key)들을 알아야 함
 - ② keySet()으로 얻어옴
 - ③ 확장 for문에서 모든 키에 해당하는 값들을 하나씩 꺼내 출력
 - Map에 저장된 객체를 꺼낼 때는 get()을 사용

3.6 application 영역(2)

- [예제 3-10]에서 저장한 속성값이 여전히 정상적으로 출력되며, 이 속성값은 웹 서버가 종료되지 않는 한 계속 유지
- 웹 서버를 재시동한 후 [예제 3-11]을 다시 실행하여 확인
 - 01 이클립스 화면 아래쪽에서 [Servers] 뷰를 클릭
 - 02 서버 이름에서 마우스 우클릭 → [Restart] 메뉴를 클릭하여 웹 서버를 재시동
 - 03 결과 확인 페이지, 즉 [예제 3-11]의 ApplicationResult.jsp 파일을 다시 실행
 - 웹 브라우저에서는 500 에러 메시지를, 이클립스의 [Console] 뷰에서는 NullPointerException을 확인

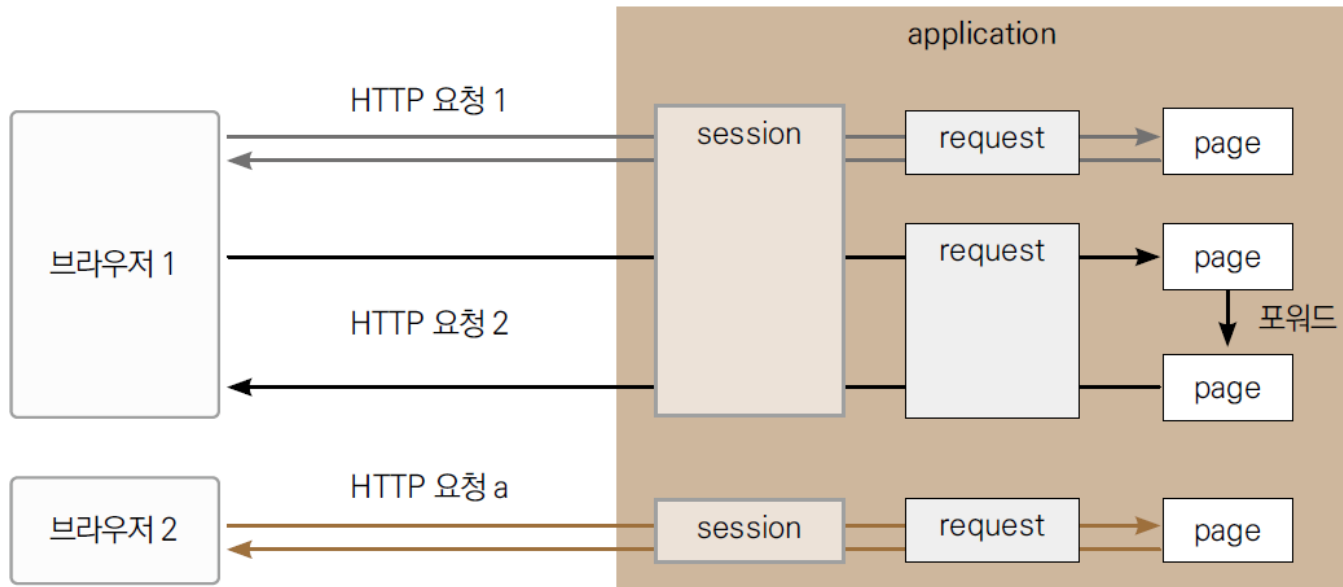
학습 마무리(1)

핵심 요약

- page 영역 : 동일한 페이지에서만 공유. 페이지를 벗어나면 소멸
- request 영역 : 하나의 요청에 의해 호출된 페이지와 포워드(요청 전달)된 페이지까지 공유. 새로운 페이지를 요청(페이지 이동)하면 소멸
- session 영역 : 클라이언트가 처음 접속한 후 웹 브라우저를 닫을 때까지 공유. 포워드나 페이지 이동 시에도 영역은 소멸되지 않음
- application 영역 : 한 번 저장되면 웹 애플리케이션이 종료될 때까지 유지. 즉, 서버가 셧다운되지 않는다면 언제까지든 공유되는 영역

학습 마무리(2)

- 내장 객체 영역별 수명주기



- session 내장객체의 주요 메서드는 다음과 같다.

[표 6-5] session 내장객체 메서드

메서드	설명
<code>getId()</code>	각 접속에 대한 세션 고유의 ID를 문자열 형태로 반환한다.
<code>getCreatingTime()</code>	세션 생성 시간을 January 1, 1970 GMT.부터 long형 밀리세컨드 값으로 반환한다.
<code>getLastAccessedTime()</code>	현재 세션으로 마지막 작업한 시간을 long형 밀리세컨드 값으로 반환한다.
<code>getMaxInactiveInterval()</code>	세션의 유지 시간을 초로 반환한다. 이를 통해 세션의 유효 시간을 알 수 있다.
<code>setMaxInactiveInterval(t)</code>	세션의 유효 시간을 t에 설정된 초 값으로 설정한다.
<code>invalidate()</code>	현재 세션을 종료한다. 세션과 관련된 값들은 모두 지워진다.
<code>getAttribute(attr)</code>	문자열 attr로 설정된 세션 값을 <code>java.lang.Object</code> 형태로 반환한다.
<code>setAttribute(name,attr)</code>	문자열 name으로 <code>java.lang.Object</code> attr를 설정한다.